

UMIで記録した動きを ロボットアームで再生できなかった理由

MuJoCoシミュレーション上のFairino FR5アームでの調査記録 / 対象ファイル:
misc/ReplayUmiOnFairino5.py

1. 何をしたかったのか

この作業のゴールはシンプルです。「人が手で持つ装置(UMI)を動かして、その動きを記録し、あとからロボットアームに同じ動きをさせる」。これができれば、ロボットに教えた動作を、ロボット本体を直接動かさずに人の手で自然に記録できます。

記録のときは、UMIの動きに合わせてシミュレーション内のアームも一緒に動かして、画面で確認しながら進めました。このときは何の問題もなく、アームはきれいに追従していました。

2. ところが再生すると全く違う動きになった

同じデータを読み込んで再生(リプレイ)させると、アームは記録とは似ても似つかない動きをし、途中で自分の腕同士がぶつかって(自己干渉)止まってしまいました。

これは非常に奇妙な状況です。記録のときに同じシミュレーションの中で問題なく動いていたのですから、その動き自体が「無理な動き」であるはずがありません。つまり原因はロボットや環境ではなく、**再生する側のプログラムにある**はずです。

3. 原因は3つありました

調べた結果、独立した3つのバグが重なっていました。順に説明します。

原因① 読み込むデータの種別を間違えていた

記録ファイルには、よく似た2種類の位置データが保存されています。

データ名	意味
command_eef_pose	「ここに動いてほしい」という指令の値
measured_eef_pose	「実際にこうなった」という結果の値

記録時のプログラムが使っていたのは**指令の値**でした。ところが再生プログラムは**結果の値**を読んでいた。この2つは実データで**最大17cmもズレ**ていました。出発点が違うのですから、違う動きになるのは当然です。

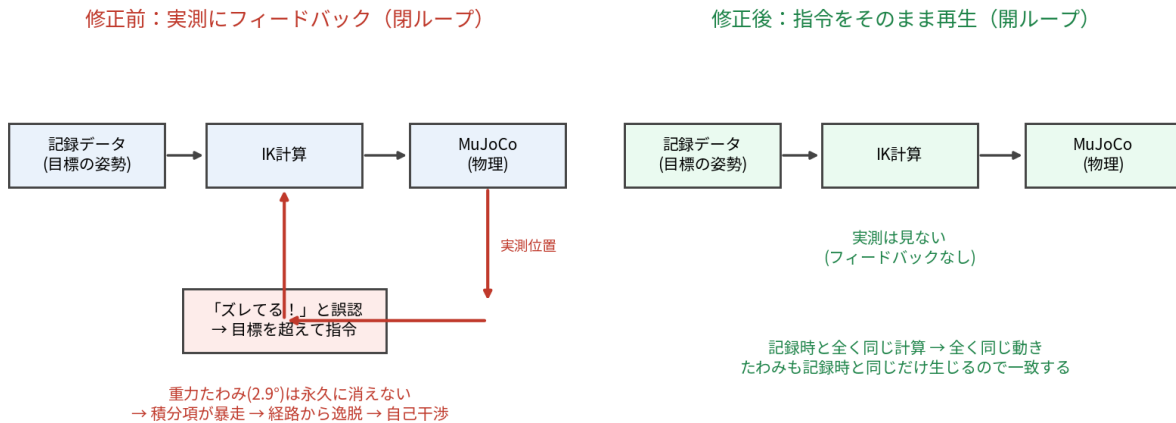
原因② 重力による「たわみ」を誤解していた（最大の原因）

これが一番わかりにくく、一番影響が大きい原因でした。

シミュレーション内のロボットの関節は、本物のモーターと同じように**重力で少したわみます**。「この角度になって」と指令しても、腕の重さで実際にはほんの少し足りない角度で止まります。実際に測ったところ、指令を出し続けて400回シミュレーションを進めても、肩の関節には**2.9度のズレが永久に残り続けました**。しかもこれは、衝突判定を全部オフにしても同じでした。つまり衝突とは無関係で、そもそも消せない性質のズレです。

記録時のプログラムは実際の位置を見ていなかったため、このたわみと共存していました。ところが再生プログラムは毎回実際の位置を確認し、このたわみを「**追従が遅れている！**」と誤解して、目標を通り越して強く指令するようになりました。しかしたわみは絶対に消えないので補正は永久に終わらず、指令はどんどん過剰になり、記録された経路から外れて、最後は**記録時には起きなかった自己干渉**に突っ込んでいました。

この誤解を図にすると



左（修正前）：実際の位置を見て補正しようとするため、絶対に消えないたわみと戦い続けて暴走する。右（修正後）：記録時と全く同じ計算だけを行い、実際の位置は見ない。たわみも記録時と同じだけ発生するので、結果として同じ動きになる。

原因③ データを勝手に加工していた

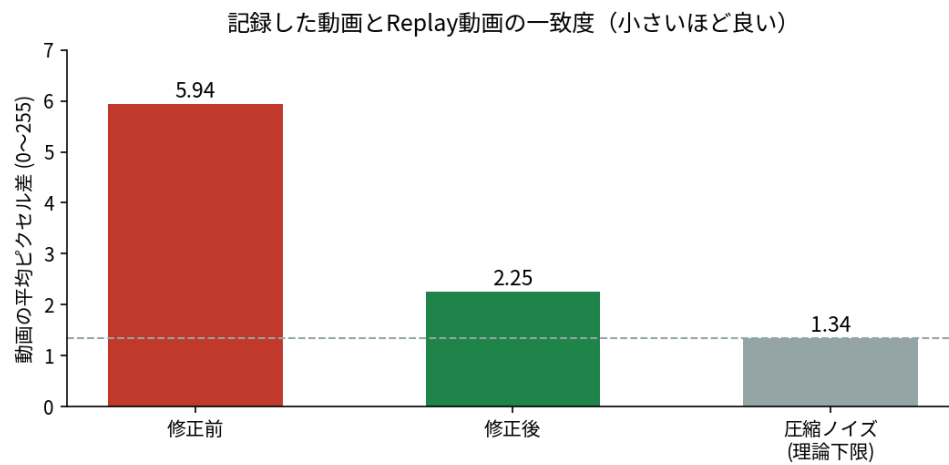
再生プログラムには、記録の最初の数秒を切り捨てたり、センサーのノイズを取り除いたりする機能が入っていました。これ自体は本物のロボットを安全に動かすためには有用ですが、「記録と同じ動きを再現する」目的では、**加工した時点で記録とは別のデータになってしまいます**。特に最初の数秒を切り捨てると、動きの基準となる出発点そのものが変わってしまいます。

4. どう直したか

原因	修正内容
① データの種類	指令の値(command_eef_pose)を読むように変更
② たわみの誤解	実際の位置を見ない方式(開ループ)に変更。 記録時と全く同じ計算をそのまま再演する
③ 勝手な加工	--mirror_exact オプションで加工を無効化できるようにした

5. 本当に直ったのか確かめる

「直ったつもり」で終わらせないため、**記録時の動画と、再生した動画を1コマずつ画像として比較**しました。

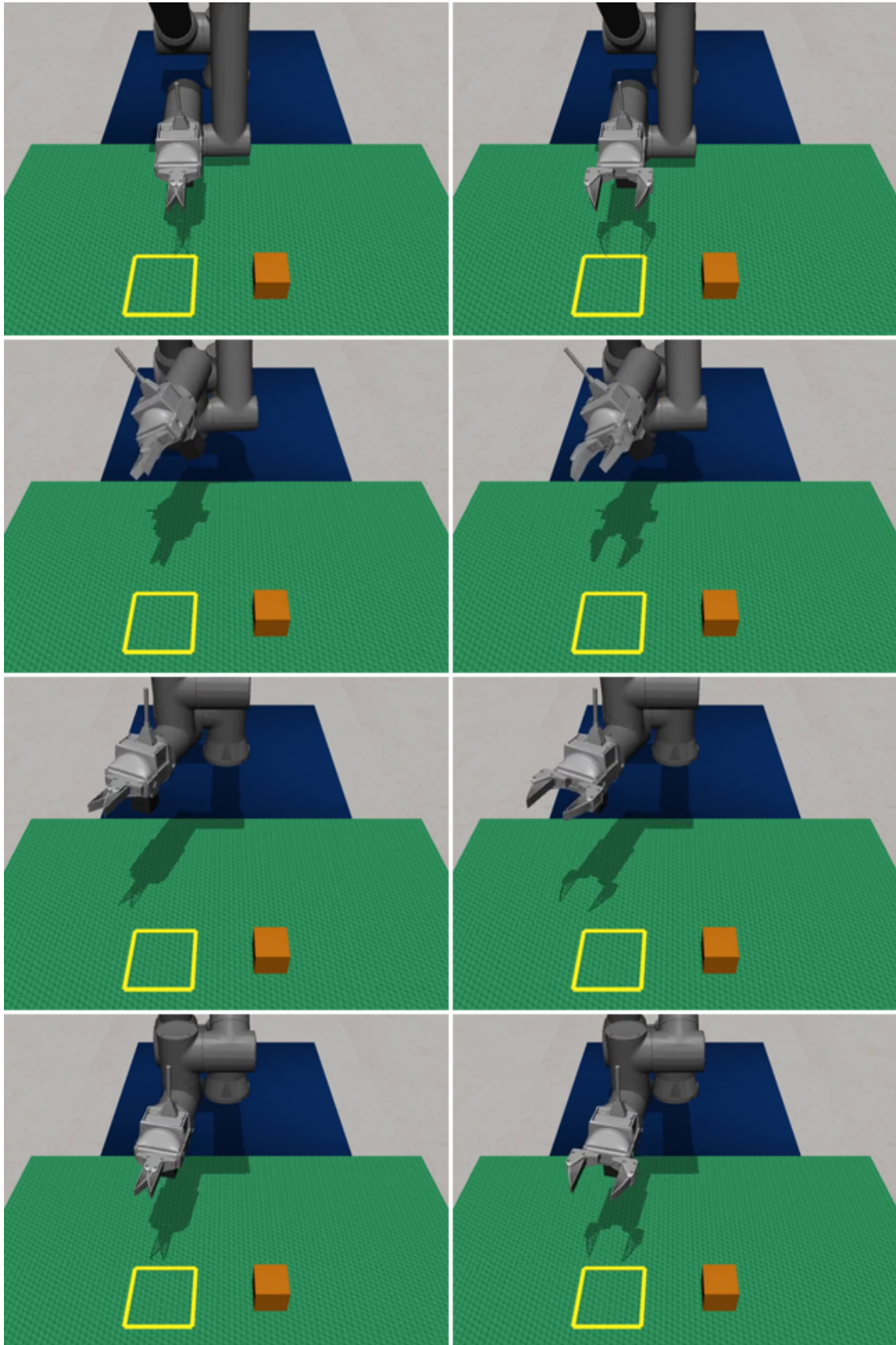


動画は圧縮して保存されるため、たとえ完全に同じ映像でも差はゼロにはなりません。その「圧縮だけによる差(1.34)」が実質的な下限です。修正後の2.25はそれにかなり近い値です。

さらにアームの位置そのものを画像から計算したところ、640×480ピクセルの画面上で、記録と再生のズレは**平均0.73ピクセル、最大でも1.57ピクセル**でした。ほぼ完全な一致です。

6. 実際の映像での比較

左が記録時、右が再生時です。どの時点でもアームの姿勢が一致しています。



左: 記録時のシミュレーション映像 / 右: 修正後の再生映像

7. この件から学べること

「賢く補正する」ことが正解とは限りません。今回の再生プログラムは、実際の位置を確認して誤差を補正するという、一見とても丁寧で正しそうな作りになっていました。しかし目的が「記録と同じ動きを再現すること」である以上、正解は**記録時と全く同じ計算をそのまま繰り返すこと**でした。補正機能はむしろ邪魔をしていたのです。

「動いているように見える」は**証拠になりません**。記録時のプログラムは実際の位置を確認していなかったため、実は同じたわみも同じ接触も起きていたのに、それに気づけませんでした。問題が「再生時だけ」に見えていたのは、単に再生側だけが正直に現実を確認していたからです。

最後は必ず目で確かめる。今回は数値だけでなく、記録と再生の動画を実際に1コマずつ比較して初めて「本当に一致した」と判断できました。

8. 使い方

```
python ./misc/ReplayUmiOnFairino5.py \  
./dataset/<DATASET_DIR>/<FILE>.rmb \  
--vive_config ./teleop/configs/ViveUML.yaml \  
--mirror_exact --sim
```

実行すると再生の映像が `_replay_sim.mp4` として保存され、記録時の `_mujoco_mirror.mp4` とそのまま見比べられます。

注意: `pos_scale` は動きの大きさ(並進)にだけ掛かり、回転には掛かりません。記録時と違う値を指定すると再生は一致しくなくなります。また、本物のロボットでの動作確認はまだ行っていません。